

ui.link 全面详细阐述

在 NiceGUI 框架中，`ui.link` 是用于创建超链接的核心 UI 组件，支持跨页面导航、页内锚点跳转、自定义点击元素等多种场景，具备灵活的目标绑定、样式定制和响应式交互能力，是构建界面导航逻辑的关键组件。以下从核心定义、基础使用、高级特性、属性与方法等维度进行全面解析。

一、核心定义与作用

`ui.link` 的核心功能是**创建可点击的超链接**，实现用户从当前界面到目标地址的跳转。其目标支持多种类型：外部 URL、内部页面路径、页内锚点、组件实例等，同时支持嵌套子元素（如图像、图标）实现自定义点击区域，本质是对 HTML `<a>` 标签的增强封装，结合 Vue 的响应式特性和 NiceGUI 的组件生态，满足各类导航需求。

二、基础使用

1. 最简示例：外部 URL 跳转

通过指定显示文本和目标 URL，快速创建指向外部资源的链接，适用于跳转至第三方网站、文档等场景：

```
from nicegui import ui

# 创建指向GitHub的外部链接
ui.link('NiceGUI on GitHub', 'https://github.com/zauberzeug/nicegui')

ui.run()
```

运行后界面显示蓝色可点击文本，点击后在当前标签页打开目标 URL（默认不新建标签页）。

2. 新建标签页跳转

通过 `new_tab=True` 参数，设置链接在新浏览器标签页中打开，避免覆盖当前页面：

```
# 新标签页打开外部链接
ui.link('NiceGUI文档', 'https://nicegui.io/documentation', new_tab=True)
```

三、核心导航场景

1. 页内锚点跳转

针对长页面场景，通过 `ui.link_target` 定义锚点，结合 `ui.link` 实现页内快速定位，适用于目录导航、长表单跳转等：

```
from nicegui import ui

# 创建导航栏（固定在顶部）
navigation = ui.row().classes('fixed top-0 left-0 right-0 bg-white p-2 shadow-sm')

# 定义页内锚点A（通过名称标识）
ui.link_target('target_A')
ui.label('')
```

```

        锚点A区域: Lorem ipsum dolor sit amet, consectetur adipiscing elit,
        sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
    ''').classes('mt-10 p-4')

# 定义锚点B（直接绑定组件实例）
label_B = ui.label(''
    锚点B区域: Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut
    aliquip ex ea commodo consequat.
    Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat
    nulla pariatur.
'').classes('p-4')

# 导航栏添加页内跳转链接
with navigation:
    ui.link('跳转到A', '#target_A') # 通过锚点名称跳转
    ui.link('跳转到B', label_B)     # 直接绑定组件实例跳转

ui.run()

```

- 核心逻辑: `ui.link_target("name")` 定义锚点名称, `ui.link` 目标设为 `#name` 即可跳转; 也可直接将组件实例作为目标, 自动定位到该组件位置。

2. 跨页面导航

支持跳转至应用内其他页面, 目标可指定为页面路径或页面函数引用, 适用于多页面应用的导航逻辑:

```

from nicegui import ui

# 定义目标页面（路径: /some_other_page）
@ui.page('/some_other_page')
def other_page():
    ui.label('这是另一个页面')
    ui.link('返回首页', '/') # 跳转回首页

# 定义首页（默认路径: /）
@ui.page('/')
def home_page():
    ui.label('欢迎来到首页, 点击下方链接跳转')
    ui.link('通过路径跳转', '/some_other_page') # 按路径指定目标
    ui.link('通过函数引用跳转', other_page)     # 按函数引用指定目标

ui.run()

```

- 优势: 通过函数引用跳转无需记忆路径, 且支持代码重构时自动更新, 降低维护成本。

3. 自定义点击元素（嵌套子组件）

通过 `with ui.link()` 语法嵌套子元素, 使整个子元素成为可点击链接, 适用于图片链接、图标链接、卡片链接等场景:

```
from nicegui import ui

# 图片链接: 点击图片跳转至GitHub
with ui.link(target='https://github.com/zauberzeug/nicegui', new_tab=True):
    ui.image('https://picsum.photos/id/41/640/360').classes('w-64 rounded-lg shadow-md')

# 卡片链接: 点击卡片 (包含文本+图标) 跳转
with ui.link(target='/other_page'):
    ui.card().classes('w-48 p-4 cursor-pointer hover:shadow-lg transition-shadow')
    ui.icon('info').classes('text-blue-500')
    ui.label('查看详情')

ui.run()
```

- 支持嵌套任意 NiceGUI 组件（如 `ui.avatar`、`ui.row`、`ui.label` 等），点击嵌套元素的任意位置都会触发跳转。

四、核心属性

`ui.link` 继承自 NiceGUI 的基础 `Element` 类，拥有以下核心属性（部分为 `BindableProperty`，支持响应式绑定）：

属性名	类型	说明
<code>text</code>	<code>BindableProperty</code>	链接的显示文本，支持双向绑定，值更新时界面自动刷新
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类（支持 Tailwind/Quasar 样式类），用于定制链接外观（颜色、大小等）
<code>client</code>	<code>Client</code>	组件所属的客户端实例，用于前后端通信关联
<code>html_id</code>	<code>str</code>	HTML DOM 中的元素 ID（2.16.0 版本新增），用于直接操作 DOM 元素
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除，只读属性
<code>is_ignoring_events</code>	<code>bool</code>	组件是否正在忽略事件，只读属性
<code>parent_slot</code>	<code>Slot None</code>	组件所属的父插槽（可设置），用于复杂布局中的插槽管理
<code>props</code>	<code>Props[Self]</code>	组件的 HTML 属性（Quasar props），用于扩展链接特性（如禁用、下划线等）
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式，用于精细化外观控制
<code>visible</code>	<code>BindableProperty</code>	组件的可见性，支持响应式绑定，值为 <code>True</code> 时显示， <code>False</code> 时隐藏

属性使用示例

```
# 定制链接样式与属性
link = ui.link('定制链接', 'https://nicegui.io')
link.classes('text-xl font-bold text-green-600 hover:text-green-800') # 字体、颜色、hover效果
link.style('text-decoration: none; margin: 10px 0;') # 移除下划线、设置外边距
link.html_id = 'custom-link' # 设置DOM ID
link.props('disabled') # 禁用链接（不可点击）
```

五、核心方法

`ui.link` 提供了丰富的方法用于文本操作、绑定、样式控制、事件处理等，以下是常用核心方法分类解析：

1. 文本与目标操作方法

方法名	参数说明	功能描述
<code>set_text</code>	<code>text: str</code> : 新文本内容	直接设置链接的显示文本，覆盖原有内容
<code>bind_text</code>	目标对象、目标属性名、正向 / 反向转换函数等	双向绑定文本：链接文本与目标对象属性相互同步（如输入框值更新链接文本）
<code>bind_text_from</code>	目标对象、目标属性名、反向转换函数等	单向绑定（从目标到链接）：目标对象属性更新时，链接文本自动同步
<code>bind_text_to</code>	目标对象、目标属性名、正向转换函数等	单向绑定（从链接到目标）：链接文本更新时，目标对象属性自动同步

文本绑定示例

```
from nicegui import ui

# 双向绑定：输入框修改链接文本
text_model = {'link_text': '初始链接文本'}
ui.input('修改链接文本').bind_value(text_model, 'link_text')
ui.link('', 'https://nicegui.io').bind_text(text_model, 'link_text')

# 单向绑定（从目标到链接）：格式化文本
status = {'version': '3.0.0'}
ui.link('', 'https://nicegui.io/docs').bind_text_from(
    status, 'version', backward=lambda x: f'查看v{x}文档'
)

# 直接修改文本
link = ui.link('旧文本', 'https://nicegui.io')
link.set_text('新文本') # 覆盖为“新文本”

ui.run()
```

2. 可见性绑定方法

用于控制链接的显示 / 隐藏，支持响应式绑定，适用于根据用户权限、状态动态显示导航链接：

方法名	核心参数	功能描述
<code>set_visibility</code>	<code>visible: bool</code> ：是否可见	直接设置链接可见性
<code>bind_visibility</code>	目标对象、目标属性名、匹配值等	双向绑定可见性：链接与目标对象属性相互同步
<code>bind_visibility_from</code>	目标对象、目标属性名、匹配值等	单向绑定（从目标到链接）：目标属性满足条件时显示链接
<code>bind_visibility_to</code>	目标对象、目标属性名、转换函数等	单向绑定（从链接到目标）：链接可见性同步到目标对象属性

可见性绑定示例

```
from nicegui import ui

model = {'is_admin': False, 'show_link': True}

# 开关控制链接显示
ui.switch('显示链接', value=True).bind_value(model, 'show_link')
ui.link('动态显示链接', '/admin').bind_visibility(model, 'show_link')

# 权限控制：仅管理员可见
ui.checkbox('管理员模式').bind_value(model, 'is_admin')
ui.link('管理员后台', '/admin_panel').bind_visibility_from(model, 'is_admin', value=True)

ui.run()
```

3. 样式与外观定制方法

方法名	核心参数	功能描述
<code>classes</code>	<code>add/remove/replace/toggle</code> ：样式类操作	新增、删除、替换或切换 HTML 样式类（支持 Tailwind/Quasar 类）
<code>default_classes</code>	同 <code>classes</code> ，但作用于类级（所有实例共享）	定义该类所有链接的默认样式类（需在实例化前调用）
<code>style</code>	内联 CSS 样式字符串	设置组件内联样式，优先级高于 <code>classes</code>
<code>default_style</code>	同 <code>style</code> ，作用于类级	定义该类所有链接的默认内联样式
<code>tooltip</code>	<code>text: str</code> ：提示文本	为链接添加鼠标悬浮提示（如说明跳转目标）

样式定制示例

```
from nicegui import ui

# 类级默认样式: 所有CustomLink实例默认加粗、蓝色文本、无下划线
class CustomLink(ui.link):
    default_classes(replace='font-bold text-blue-600 no-underline')
    default_style(add='transition: color 0.3s;') # 颜色过渡动画

# 实例级样式修改
link1 = CustomLink('类默认样式', 'https://nicegui.io')
link2 = CustomLink('实例自定义样式', 'https://nicegui.io/docs')
link2.classes(add='hover:text-blue-800') # hover时加深颜色
link2.style('margin-top: 10px;') # 上外边距
link2.tooltip('查看官方文档') # 悬浮提示

ui.run()
```

4. 事件与交互方法

方法名	核心参数	功能描述
on	事件类型、处理函数、JS 处理函数等	绑定事件（如 click、mousedown），支持 Python/JS 双端处理（跳转前触发逻辑）
mark	标记字符串（单个或多个）	为组件添加标记，用于测试查询或依赖管理
update	无参数	强制更新组件，同步后端状态到前端

事件绑定示例（跳转前触发逻辑）

```
from nicegui import ui

# 点击链接前触发确认弹窗
def confirm_navigation():
    ui.dialog().open()
    with ui.card():
        ui.label('确定要跳转吗?')
        with ui.row():
            ui.button('取消', on_click=lambda: ui.close_modals())
            ui.button('确定', on_click=lambda: (ui.close_modals(),
link.run_method('click'))))

link = ui.link('点击跳转（需确认）', 'https://github.com/zauberzeug/nicegui')
# 覆盖默认点击事件，先触发确认逻辑
link.on('click', lambda e: (e.preventDefault(), confirm_navigation()), js_handler='(e) => e.preventDefault()')

ui.run()
```

- 核心逻辑: 通过 `e.preventDefault()` 阻止默认跳转，触发自定义逻辑（如确认弹窗），确认后通过

`run_method('click')` 执行原跳转。

5. 布局与容器相关方法

方法名	核心参数	功能描述
<code>add_slot</code>	插槽名、Vue 模板	为组件添加 Vue 插槽，用于复杂子元素布局（如自定义嵌套结构）
<code>move</code>	目标容器、目标索引、目标插槽	将链接移动到其他容器或插槽中（如动态调整导航栏顺序）
<code>clear</code>	无参数	删除链接的所有子元素（如嵌套的图片、文本）
<code>remove</code>	子元素实例或 ID	删除指定子元素
<code>delete</code>	无参数	删除链接自身及所有子元素

6. 资源与依赖管理方法

方法名	核心参数	功能描述
<code>add_resource</code>	资源路径（文件夹 / 文件）	为链接添加静态资源（如自定义 CSS/JS 文件，用于增强样式或交互）
<code>add_dynamic_resource</code>	资源名、生成函数	为链接添加动态资源（函数返回资源响应，如动态生成跳转目标）
<code>get_computed_prop</code>	属性名、超时时间	异步获取组件的计算属性（需 <code>await</code> ，如获取链接的实际跳转 URL）
<code>run_method</code>	方法名、参数、超时时间	调用客户端侧方法（如触发点击、获取 DOM 属性）

六、高级特性

1. 响应式目标绑定

结合 `bind_*` 系列方法，可实现跳转目标的动态更新，适用于根据状态切换跳转地址：

```
from nicegui import ui

# 动态切换跳转目标
model = {'env': 'prod'}
def toggle_env():
    model['env'] = 'dev' if model['env'] == 'prod' else 'prod'
    link.update()

ui.button('切换环境', on_click=toggle_env)
link = ui.link('访问API', 'https://api.prod.com')

# 绑定目标URL：根据env动态切换
```

```
link.bind_text_from(model, 'env', backward=lambda x: f'访问{x.upper()}环境API')
link.bind_prop('target', model, 'env', backward=lambda x: f'https://api.{x}.com')

ui.run()
```

2. 禁用与状态控制

通过 `props('disabled')` 或绑定可见性，实现链接的禁用状态，适用于未满足条件时禁止跳转（如未登录、表单未完成）：

```
from nicegui import ui

model = {'form_completed': False}

# 表单完成后启用链接
ui.checkbox('我已完成表单', value=False).bind_value(model, 'form_completed')
link = ui.link('提交表单', '/submit')
# 表单未完成时禁用链接
link.bind_prop('props', model, 'form_completed', backward=lambda x: ' ' if x else 'disabled')
link.classes('opacity-50' if not model['form_completed'] else 'opacity-100')

ui.run()
```

3. 自定义跳转逻辑

通过 `on('click')` 事件覆盖默认跳转行为，实现复杂逻辑（如日志记录、权限校验、异步操作）后再跳转：

```
from nicegui import ui
import asyncio

# 异步校验权限后跳转
async def check_permission_and_navigate():
    ui.notify('正在校验权限...')
    await asyncio.sleep(1) # 模拟异步校验
    ui.notify('权限通过，即将跳转')
    link.run_method('click') # 执行原跳转

link = ui.link('权限校验后跳转', 'https://admin.nicegui.io')
link.on('click', lambda e: (e.preventDefault(),
    asyncio.create_task(check_permission_and_navigate()))),
    js_handler='(e) => e.preventDefault()')

ui.run()
```

七、版本兼容性说明

部分属性和方法存在版本限制，使用时需注意：

- `html_id`：2.16.0 版本新增；
- `default_classes` 的 `toggle` 参数：2.7.0 版本新增；

- `on` 方法支持同时指定 Python 和 JS 处理函数：2.18.0 版本更新；
- `strict` 参数（绑定方法中）：3.0.0 版本新增，用于校验目标对象的属性是否存在。

八、适用场景总结

1. 外部资源跳转：链接到第三方网站、文档、下载地址等；
2. 内部页面导航：多页面应用中实现页面间切换（如首页→详情页、列表→编辑页）；
3. 页内锚点定位：长页面目录导航、表单分区跳转、滚动定位；
4. 自定义点击元素：图片链接、卡片链接、图标链接等非文本点击区域；
5. 动态导航控制：根据用户权限、状态动态显示 / 隐藏链接，或切换跳转目标；
6. 带逻辑跳转：跳转前执行确认、日志记录、权限校验等自定义逻辑。

通过上述特性，`ui.link` 成为 NiceGUI 中功能灵活的导航组件，既能满足简单的文本链接需求，也能通过嵌套、事件绑定等高级特性实现复杂的交互导航逻辑，是构建用户友好界面的核心组件之一。